十年老技术开发的 AI Agent 探索之路

腾讯程序员 腾讯技术工程 2026年5月7日 02:34 广东 116人



作者：zhiyuanfu

> 曾经前端被戏称为"娱乐圈"——工具、框架层出不穷，今年🔥 的明年就过时。现在 AI 把这个周期压缩到了以月计：这个月的新概念，下个月可能就是旧闻。这篇文章，就是一个在"AI 娱乐圈"摸爬滚打的老开发，试图从月抛式的焦虑中找到不会过期的东西，为大家抛砖引玉。

- 4-6 个终端的并发上限，怎么突破
- 80% 的 AI 需求，10 行 Bash 就够了
- Vibe Coding 翻车全记录
- 24h 无人值守的代码开发 Agent 怎么造
- 从 Task-Driven 到 Goal-Driven 的认知跃迁

第一章：起点——人是瓶颈

此刻我的屏幕上开着 5 个终端。

左上角，codex 正在跑一组单元测试，终端里绿色的 pass 和偶尔的红色 fail 交替滚动。右上角，gemini-cli 在按照我刚给的方案改一个接口的入参校验。左下角，claude 在根据最新的 API 变更生成文档。右边整块屏幕留给了 Cursor，里面同时开着两个 Agent 窗口——一个在重构组件，一个在补集成测试。

看起来很酷？

真实体验是这样的：codex 那个终端跑了五分钟没动静，我得翻上去看是卡住了还是在等确认；gemini 改完接口了，但我忘了它改的是哪个分支；claude 写的文档引用了一个旧接口名，因为我忘了告诉它刚才 gemini 改过了；Cursor 里的重构窗口弹了个确认框，我一直没注意到，白白等了十分钟。

这种模式的上限大概就是 4-6 个并发。再多，人脑的 context switch 就开始崩溃。

人工并发有三个硬伤：

限制	具体表现	后果
吞吐有限	一天能管 4-6 个 Agent 窗口	任务量有硬上限
状态不稳定	上下文丢失、判断漂移、质量波动	上午管 5 个，下午犯困管不了 3 个
难以规模化	做成一次不难，稳定重复难	今天的成功经验明天就忘了

所以真正的命题不是"怎么让 AI 更聪明"。Agent 的价值不是替人做事，是把依赖人的高频工作，改造成可以持续执行、可观测、可复盘、可优化的系统。

人是瓶颈。但解决瓶颈的方式不是让 AI 替代人，而是让系统不再依赖人的实时在场。

想明白这件事之后，我开始动手。但在造系统之前，我先学到了一条最重要的原则。

> **工程建议：** 如果你现在也在手动管多个 AI 终端，先别急着造系统。先记录一周：哪些操作是重复的？哪些切换是可以消除的？瓶颈清单比技术方案更重要。

你现在同时开几个 AI 窗口？上限是多少？评论区聊聊。

第二章：80% 的 AI 需求不需要 AI

我开始认真折腾 AI 的时候，第一件事不是去调模型、搞 RAG，而是写了一套 Bash 脚本来自动化日常工作流。

结果发现——**80% 的"AI 需求"，根本不需要 AI。**

自动拉取代码跑测试？Bash。定时检查服务健康状态？`cron + curl`。把 JSON 日志格式化成报表？`jq + awk`。文件变更触发构建？`inotifywait + shell`。这些东西不需要任何模型，10 行脚本就搞定了。

```bash
#!/bin/bash
# 例：定时健康检查 + 告警，不需要任何 AI
while true; do
  status=$(curl -s -o /dev/null -w "%{http_code}" https://api.example.com/health)
  if [ "$status" != "200" ]; then
    curl -X POST "$WEBHOOK_URL" \
      -d "{\"msg\": \"API health check failed: HTTP $status\"}"
  fi
  sleep 300
done
```

这让我提炼出后来最重要的一条原则：

> 代码优先于 Prompt。能用 10 行 Bash 解决的，别折腾 AI。

听起来像废话？但你去看看市面上多少项目，明明一个 `cron + curl` 就能搞定的定时数据采集，非要套一层 LangChain，加个 Agent 循环，搞个 tool calling，最后效果还不如写死的脚本稳定。

这个认知后来演化成了一个决策层级：



目标 → 代码 → CLI → Prompt → Agent。

层级	适用场景	示例	不确定性
目标层	想清楚到底要解决什么	想清楚后发现不需要写代码	最低
代码层	确定性逻辑	if/else、正则、模板引擎	低
CLI 层	组合现有工具	`grep + jq + curl` 串流程	中低
Prompt 层	需要语义理解和判断	需求翻译、文案生成	中高
Agent 层	多步推理、动态决策、循环执行	自动修 bug、端到端流程	最高

每往上一层，不确定性增加一个量级，成本也增加一个量级。原则很简单：**能在下层解决的，绝不上推。**

能用 10 行 Bash 解决的，别折腾 AI。这不是反 AI，是尊重工程。

> **工程建议：** 拿到一个新需求时，从表格最底行往上看——先问"10 行 Bash 能搞定吗？"，再问"一次 LLM 调用够吗？"，最后才考虑 Agent。这个习惯会帮你省掉 80% 的过度工程。

你团队里有没有"明明脚本就能搞定，偏要上 AI"的项目？说出来让大家乐乐。

第三章：Vibe Coding 翻车记

知道了"什么时候该用 AI"，接下来就是动手造系统了。

但在造出正经系统之前，我先翻了一次车。

24h 打工人项目初期，我也尝试过 Vibe Coding：不写 spec、不做设计，直接跟 AI 说"帮我做个 XXX"，然后看着它一顿操作猛如虎。

下面是真实时间线：

```
Day 1-3   ✨ "wow 这 AI 真厉害"
             几句话出一个完整页面，说需求就能跑通流程
             产出速度惊人，成就感爆棚

Day 7     ⚠️ 代码开始乱了
             AI 对功能的实现越来越差
             陷入"打地鼠"——修了这个 bug 冒出那个
             告诉它"这里有问题"，它改了之后引入两个新问题

Day 14    🔥 被迫亲自打开每个文件浏览
             大量过度设计、冗余逻辑
             三层抽象解决一个本该一个函数搞定的问题
             重复的工具方法散落在五六个文件里

Day 15    🔧 整整一天"设计与实现对齐"
             把 AI 写的代码和手写的设计文档一一对照
             逐个重构，这一天比前两周加起来都累
             但这一天的价值，也比前两周加起来都大
```

Vibe Coding 的问题本质很简单：它是"先易后难"。前期省掉的设计时间，后期会以 10 倍的 debug 时间还回来。代码越写越多，AI 的上下文越来越混乱，每一次修改都在给系统埋雷。

SDD[Q] 恰好相反。写 spec 很慢，做设计很枯燥，但一旦 spec 写清楚了，后面的执行、验证、迭代全都有据可循。

大路平坦宽阔，但人偏偏喜欢走捷径。Vibe Coding 就是那条看起来省事的小路——走着走着就发现，路越来越窄，荆棘越来越多，最后还得退回来走大路。

Vibe Coding 是先易后难。SDD 是先难后易。大道如夷，而民好径。

Day 15 那一天的"设计与实现对齐"很痛苦。但正是这一天，建立了让系统后续能自动运转的全部基础——设计文档、架构约束、SDD 流程。没有这一天，就没有后面的 24h 打工人。

> **工程建议：** 如果你现在正在 Vibe Coding，享受前几天的快感没问题，但第三天就要开始补 spec。越早补，代价越小。哪怕只有三段话——要做什么、不做什么、怎么算完成。

你 Vibe Coding 翻车过吗？最后是怎么收场的？

第四章：24h 打工人——第一个真正的系统

翻车之后，我重新来过。这一次，先设计再动手。

场景是这样的：用户提了个 bug——"搜索结果列表的分页有问题，切换页码后数据没更新"。半小时后，AI 自动完成了分析需求、生成技术方案、拆解任务、并发执行前后端代码修改、通知我 review。

不是 demo。不是手动跑了五遍调通的演示。是一个真正能 24 小时无人值守运行的系统。

我叫它 **24h 打工人**。

为什么选 CLI 而不是 API

先说一个很多人会问的问题。

答案不是教条，是阶段性选择。在我当时的场景里，codex、gemini-cli、claude-code 这些工具已经内置了读文件、改代码、跑命令的能力。它们本身就是完整的 Agent——有上下文管理、有工具调用、有错误处理。

我要做的不是重新造一个 Agent，而是造一个"管理 Agent 的调度层"。

> 在我当时的阶段里，CLI 是最低成本、最容易 debug、最利于 AI 直接读取和修复的方案。

这不是在论证"CLI 一定优于 API"。等哪天我需要更细粒度的控制、更低的延迟、更高的并发，会毫不犹豫切到 API。工具是手段，不是信仰。

自建调度层：核心架构

核心架构四个字就能概括：**文件 + 轮询**。

```
# 调度伪代码
class AgentScheduler:
    def __init__(self):
        self.queue = FileQueue("storage/feedbacks/")
        self.tools = ToolProber(["codex", "gemini-cli", "claude"])
        self.state = JsonStateManager("storage/state/")

    def run(self):
        while True:
            task = self.queue.poll()
            if not task:
                time.sleep(10)
                continue

            tool = self.tools.get_available()  # 自动选可用工具
            try:
                result = tool.execute(task.prompt, task.workspace)
                self.state.update(task.id, status="done", output=result)
            except QuotaExhausted:
                self.tools.cooldown(tool, duration=300)  # 5 分钟冷却
                self.queue.requeue(task)  # 重新入队，换工具执行
            except Exception as e:
                self.state.update(task.id, status="failed", error=str(e))
```

调度层做四件事：

1. **接收任务**：用户反馈进来，写入文件队列
2. **分发执行**：轮询队列，调用 CLI 执行
3. **状态管理**：记录每一步的输入输出，持久化到文件
4. **失败切换**：某个 CLI 配额用完，自动换下一个

选型极其简单：**文件系统存储 + 轮询调度 + JSON 状态管理。** 不是在论证"文件系统一定优于数据库"。对一个还在高速迭代的 Agent 系统来说，文件系统的好处很实际：出问题直接让 AI 看文件，不用查数据库；方便 Git 版本控制；本地和生产环境更一致。

SDD：让 AI 的每一步都有据可查

这套系统里最核心的概念是 **SDD（Spec-Driven Development）**。

很多人把 SDD 理解为"先写文档再开发"。但在 Agent 场景里，SDD 更重要的作用是：**把一件事从模糊想法逐步转成可执行单元，并且把这个过程完整留下来。**

每个需求处理完会留下一组文档：

```
storage/feedbacks/2026-01-15/20260115-143021-a1b2c3/
├── sdd/
```

```
|   ├── spec.md      # 规格说明：目标、验收标准
|   ├── plan.md      # 技术方案：涉及文件、实现步骤
|   └── tasks.md     # 任务清单：每个任务的描述和状态
├── tasks.json       # 任务执行状态（供程序读取）
└── debug/
    ├── prompts/     # 每一步的 prompt
    └── agent.log    # 执行日志
```

`spec.md` 把"分页有问题"变成"切换页码后列表数据未刷新，原因是查询参数未传递 page 参数"。`plan.md` 把问题变成方案。`tasks.md` 把方案拆成可执行的步骤，每一步都有明确的输入、输出和完成标准。

如果一次执行没有留下足够上下文，你就回答不了四个关键问题：

- 它当时看到了什么输入？
- 为什么做出这个判断？
- Prompt 在哪个环节失效了？
- 哪些动作已经足够稳定，可以固化成 Skill？

没有这些记录，系统就只能不断"重来一次"；有了这些记录，系统才可能真正"学会下一次做得更好"。

留痕不是为了 debug，而是为了进化。

智能并发策略

任务拆解完成后，系统按项目分组执行：

策略	具体做法	理由
组间并发	前端任务和后端任务同时跑	代码在不同目录，不会冲突
组内串行	同一个前端项目的任务排队执行	可能修改同一文件，避免冲突
失败隔离	单个任务失败不影响其他组	故障不扩散

并行的本质不是"同时做很多事"，而是"让每件事都不需要等别人"。

工具失败自动切换

AI CLI 工具经常遇到配额限制。我的方案是配合 Tool Prober 定时探测工具可用性：

```
正常流程：task → codex（可用）→ 执行成功 → 完成
失败切换：task → codex（配额耗尽）→ gemini-cli → 执行成功 → 完成
全部不可用：task → 等待 → 5 分钟后自动探活 → 恢复后继续
```

单个工具挂了不影响整体，配额耗尽自动切换。这套机制让系统真正做到了 24 小时无人值守——从 4 个终端的手忙脚乱，到 20-30 个并发任务的稳定执行。

> **工程建议：** 起步阶段，文件系统 + JSON 状态比数据库更适合 Agent 系统。原因很实际——出了 bug 可以直接让 AI 读文件排查，不需要教它查数据库。等系统稳定到需要事务和并发锁的时候，再升级不迟。

你的 Agent 系统出了 bug，排查过程是什么样的？靠翻日志还是靠猜？

第五章：Agent 自己修了自己的 bug

花了一整天做"设计与实现对齐"之后不久，一个有意思的事情发生了。

那天我在用 24h 打工人的需求澄清页面，发现了一个 bug：无法选择待确认问题的选项，也没有提交按钮。页面渲染出来了，但交互完全不能用。

以前遇到这种事，流程是：打开代码 → 定位问题 → 手动修复 → 测试 → 提交。

但这次我换了个做法——直接通过 24h 打工人自己的反馈系统提交了这个 bug。

```
[用户反馈] 需求澄清页面有 bug：无法选择待确认问题的选项，也没有提交按钮。

↓ 系统自动触发 SDD 流程

[spec.md] 目标：修复需求澄清页面的交互组件渲染问题
          验收标准：选项可点选，提交按钮可见且可用

↓ AI 生成方案并拆解任务

[plan.md] 定位：RadioGroup 组件未正确绑定 onChange 事件
          方案：修复组件 props 传递，补充提交按钮渲染逻辑

↓ 我确认澄清结果

[执行] 分析代码 → 定位问题 → 修改组件 → 验证修复 → 完成

↓ 企微通知：任务已完成
```

它自己修复了自己的 bug。

整个过程我只做了两件事：提交反馈、确认澄清。

这件事背后有一个严肃的前提：**自举不是凭空发生的。**

还记得 Day 15 花了整整一天做的"设计与实现对齐"吗？那一天的工作产出不只是修复了代码，更重要的是建立了三样东西：

1. **清晰的设计文档** —— AI 知道每个模块该做什么、不该做什么。

2. **SDD 流程** —— spec → plan → tasks 的标准路径，AI 按照同样的方式处理所有需求，包括关于自身的需求。
3. **constitution.md** —— 架构约束文件，定义了代码组织规范、命名规则、模块边界。AI 在生成方案时会自动遵循这些约束。

没有这些基础设施，AI"自己修自己"就只是碰运气。有了这些，它才能在框架内工作，而不是自由发挥。

从 Vibe Coding 的混乱，到一天的阵痛，到 Agent 自举——这条路的因果链非常清楚。

捷径的尽头是弯路，大道的尽头是自由。

> **工程建议：** 自举的前提是 constitution.md（架构约束文件）。不需要写得多长，但至少要覆盖三件事：目录结构约定、模块边界、命名规则。有了它，AI 才能在"框架内工作"而不是"自由发挥"。

你见过最惊艳的 AI 自举案例是什么？

第六章：从 demo 到系统——门槛不是模型，是治理

做完 24h 打工人之后，我慢慢意识到一件事：**留痕只是起点，不是终点。**

很多 Agent demo 的问题不是它不会跑，而是它一旦跑偏，你完全不知道发生了什么。我把这叫做"demo 跑偏时的 4 个不知道"：

1. 是目标有歧义，还是分解有问题？
2. 是工具挂了，还是权限不够？
3. 是 Prompt 不稳定，还是系统边界不清？
4. 是这次偶然成功，还是可以稳定复现？

这四个问题回答不了，demo 就永远是 demo。

Observability：6 个必看维度

我以前说"留痕很重要"。现在更愿意说：留痕是 debug 的起点，observability 才是系统优化的闭环。

一个生产级 Agent 系统，至少要看得见：

```
observability_dimensions:
  1_goal:      "当前目标是什么"
  2_step:      "正在执行哪一步"
  3_tool:      "用了什么工具"
  4_failure:   "为什么失败"
  5_recovery:  "是否触发了重试 / 回退 / 切换"
  6_cost:      "本轮 token / 时间 / 成本消耗了多少"
```

如果没有这些视图，系统一复杂，就只能靠猜。猜着猜着就不想维护了。

Eval：4 个持续校准问题

传统软件习惯把评估理解成"上线前测一下"。但 Agent 面对的是开放环境、动态输入、不断变化的上下文。评估必须是持续的：

评估问题	为什么重要
需求澄清是否稳定？	不稳定意味着下游全部白跑
任务拆解是否越来越合理？	拆解质量决定执行效率
某个 Skill 是否真的提高了成功率？	避免"加了等于没加"
失败切换有没有制造新错误？	避免"修了一个坑，挖了两个坑"

对 Agent 来说，**评估不是验收动作，而是日常运行信号。**

Control Plane：权限、边界、审计

当系统接了更多工具、更多角色、更多目标，问题就不再是"能不能跑"，而是：

- 谁可以调用哪些工具？
- 哪些动作默认允许，哪些必须人工确认？
- 哪些路径只能读，哪些能写？
- 哪些任务失败后必须停下来？

从 demo 到系统，中间隔着的不是更多 Prompt，而是 control plane。

脚手架 > 模型

这是我所有原则里最反常识的一条。

24h 打工人用的不是最贵的模型。codex 配额用完切 gemini，gemini 挂了切 claude——都不是顶配。但有 SDD 流程 + 调度层 + 失败切换 + constitution.md，效果远好于直接用一次性的顶级模型。

```
投入回报对比（个人经验估算）：

模型升级：成本 +300%，效果 +20%
脚手架升级：成本 +50%，效果 +200%

→ 优先投资脚手架，而不是追最新最贵的模型
```

一个设计精良的系统让弱模型发挥惊人性能。反过来，烂系统完全浪费掉顶级模型的能力。

Agent 系统的 5 层地基

如果今天让我重新概括 Agent 系统的地基，至少有五层：

1. **目标表达**：到底想完成什么
2. **能力单元**：有哪些 Skill 工具 工作流
3. **运行时状态**：当前正在做什么
4. **治理边界**：允许做什么，不允许做什么
5. **评估反馈**：哪些行为值得固化，哪些必须修正

少任何一层，系统都可能看起来能跑，但跑不稳。

垃圾的思考乘以强大的模型，等于精美的垃圾。

> **工程建议：** 如果你的系统还没有 observability（至少能回答"它为什么失败"），那比换一个更强的模型优先级高 10 倍。先投治理，再投模型。

你觉得"脚手架 > 模型"对吗？有没有反例？

第七章：协议层正在成形

做到这里，我的视角发生了第二次升级。

以前关心的是：怎么把一个 Agent 系统搭起来。现在更关心的是：**整个行业在形成哪些共识性的基础设施？**

如果只盯着单个 Agent，很容易把问题看成"Prompt + 工具 + 工作流"。但从 2025 年开始，一个更大的变化出现了：Agent 正在从单系统里的自动化，走向跨系统互操作。

时间	事件	意义
2025-03-11	OpenAI 发布 Agents 新基座	Responses API + Tools + SDK + Tracing，runtime 开始收敛
2025-04-09	Google 发布 A2A	Agent 间协作有了协议化趋势
2025-06-23	A2A 捐给 Linux Foundation	从企业项目变行业标准
2026-02-13	GitHub 发布 Agentic Workflows 技术预览	Agent 进入 CI PR Issue 主流程
2026-03-16	Microsoft Foundry Agent Service GA	Responses API runtime + 全链路 tracing + 企业级 eval
2026-04-07	GitHub Dependabot → AI agent remediation	安全告警自动修复成为现实

Responses API：runtime 在收敛

OpenAI 把 Responses API、内置 Tools、Agents SDK、Tracing 组合成了更明确的开发路径。Assistants API 后续将逐步让位。

这件事的重要性在于：Agent 开发正在从"自己拼一堆能力"，走向"围绕统一 runtime 和工具语义来构建"。

MCP：工具接入标准化

我自己过去解决的是"怎么让 Agent 调 CLI、读文件、改代码"。MCP 解决的是更通用的问题：Agent 怎么以标准方式接入工具、资源和外部系统。

```
{
  "tool": "code_search",
"description": "Search codebase by semantic query",
"input_schema": {
    "type": "object",
    "properties": {
      "query": { "type": "string" },
      "scope": { "enum": ["current_repo", "org_repos"] },
      "max_results": { "type": "integer", "default": 10 }
    }
  },
"permissions": ["read:code"],
"rate_limit": "100/min"
}
```

一旦工具接入开始标准化，重点就从"每家自己造一套胶水层"转向"怎样设计更稳定的资源暴露、权限边界和调用语义"。

A2A：多 Agent 怎么协作

Agent 的下一个问题，不是单点更聪明，而是多 Agent 怎么协作、发现、协商和同步状态。

这和我自己踩过的坑是同一类问题：主 Agent 上下文越来越重；子 Agent 返回后如何续跑；多角色如何共享状态而不是靠人手工搬运上下文。以前只能各自造轮子，现在开始出现协议化趋势了。

我的判断

> Agent 开发正在从"框架之争"，转向"协议 + runtime + control plane 之争"。

这不代表框架不重要，而是它们正在往更底层、更标准化的方向收敛。对个人开发者来说，现在值得花时间理解的不是某个框架的 API，而是这些协议背后的设计理念。

未来做 Agent，越来越像搭操作系统，不只是写 prompt。

> **工程建议：** 选技术栈时，优先看它是否兼容 MCP / Responses API 这些正在收敛的协议。自己造的胶水层越多，未来迁移成本越高。协议层是长期资产，框架是短期工具。

你在用 MCP 了吗？体验怎么样？

第八章：从 Task-Driven 到 Goal-Driven

前面讲的所有实践，回头看，本质上都是 **Task-Driven**：我先把值得做的事想好，送进系统，系统负责拆解、执行、留痕。

这套模式非常有效。但它有一个我撞了很久才意识到的边界：

24h 在线，不等于 24h 迭代。

系统虽然能 24 小时执行，但这些更高层的判断仍然依赖我：

- 现在最值得做什么？
- 哪个方向应该继续推进？
- 遇到阻塞时该换路径还是等待？
- 哪些问题值得主动探索？

系统在干活，但活还是我在派。**只要任务还需要我持续供给，我就仍然是系统的瓶颈。**

这才是问题的根。

Task-Driven vs Goal-Driven



维度	Task-Driven	Goal-Driven
人的角色	项目经理 + 执行监督	目标设定者 / 审核者
Agent 的角色	执行器	自主推进者
决策中心	在人脑子里	在目标 + 边界 + 系统状态里

维度	Task-Driven	Goal-Driven
主要成本	人持续编排	前期建模和约束设计
适用场景	简单、一次性任务	长期、复杂、持续推进任务

简单说：**Task-Driven 解决执行问题，Goal-Driven 解决迭代问题。** 前者让系统开始能跑，后者才让系统开始能持续向前。

Goal-Driven 的 5 个前提

很多人不敢放手让 Agent 自主推进，担心它跑偏、浪费 token、做无效工作。这些担心都成立。

所以 Goal-Driven 对系统提出了更高要求：

1. **目标必须清晰** —— 不是模糊愿望，而是可推进、可判断的目标表达。
2. **边界必须清晰** —— 哪些能做，哪些不能做，资源上限是什么。
3. **状态必须可见** —— 当前做到哪一步，卡在哪，为什么卡。
4. **过程必须留痕** —— 否则你无法知道它为什么成功，也无法知道它为什么失败。
5. **权限必须可控** —— 它到底能调用哪些工具，能写到哪里，谁来兜底。

只有在这 5 个前提成立时，自主推进才是资产；否则只会把错误放大得更快。

Goal-Driven 不是更放权，而是更强约束下的有限自治。

共享状态：STATE.yaml

Goal-Driven 一旦进入多步骤、多角色协作，主 Agent 本身会变成新的瓶颈——上下文越来越重、通信越来越慢、单点故障。

更务实的做法是用共享状态来协调：

```yaml
# STATE.yaml - 共享任务面板
goal: "优化搜索模块响应速度，P95 从 800ms 降到 200ms"
owner: "joefu"
deadline: "2026-05-01"
constraints:
  - "不修改已有 API 契约"
  - "每日 API 成本不超过 $5"
  - "所有架构变更需记录在 changelog.md"

agents:
  profiler:
    status: "completed"
    output: "analysis/search-perf-baseline.json"
    summary: "瓶颈定位在 DB 全表扫描和缺少缓存层"
```

```yaml
backend_dev:
    status: "in_progress"
    current_step: "为热点查询添加 Redis 缓存"
    blocked: false

test_runner:
    status: "pending"
    depends_on: ["backend_dev"]
    description: "回归测试 + 性能压测验证"

next_review: "2026-04-15"
```

每个 Agent 自己读取状态、写回进度，主会话只负责高层目标和验收。主会话负责方向，不负责搬运；系统负责推进，不靠人反复传话。

6 步落地路径

如果你也想开始做 Agent，我建议这个顺序：

步骤	做什么	核心产出
第一步	写清楚 spec	要做什么、不做什么、怎么算完成
第二步	执行过程留痕	Prompt 状态 输出 / 错误全记录
第三步	补 observability 和 eval	知道为什么成功、为什么失败
第四步	高频动作沉淀为 Skill	模板 + 规则 + 代码
第五步	引入调度和并发	调度层 + 轮询 + 失败切换
第六步	最后才尝试 Goal-Driven	目标表达 + 治理边界 + 共享状态

先让一次执行可复盘，再让它可重复，再让它可规模化，最后让它可有限自主。

> **工程建议：** Goal-Driven 必须建立在成熟的 Task-Driven 基础上。一个连任务都执行不稳定、没有留痕、没有 Skill 沉淀的系统，不可能真的进入目标驱动。别跳步。

你的 Agent 系统现在在哪一步？卡在哪里？

结语：增强自我，而非取代自我

回到开头的问题：人是瓶颈。

但一年走下来，我对这句话的理解变了。

瓶颈不是人的能力不够，而是人的注意力有限。4-6 个终端是上限，不是因为不够努力，而是人脑的并发模型就长这样。解决方案不是"让 AI 替代人"，而是"让系统不再依赖人的实时在场"。

24h 打工人的 SDD 是这个系统的地基：

- `spec.md` 把模糊的需求变成明确的目标
- `plan.md` 把目标变成技术方案
- `tasks.md` 把方案变成可执行的步骤
- `constitution.md` 把经验变成可复用的约束
- `eval / trace / policy` 把系统变成可观测、可治理、可持续迭代的能力体

Goal-Driven 是它的下一站：让系统不只是等你派活，而是围绕目标自主向前走。

回顾整条路——从 4 个终端的手忙脚乱，到 Vibe Coding 翻车后的痛定思痛，到 24 小时无人值守的稳定执行，到 Agent 自己修自己的 bug，再到协议层正在收敛的行业共识——每一步的认知转折都不是提前设计好的，是被实践逼出来的。

真正的跃迁，不是让 AI 多做几个步骤，而是让人退出微观调度。

增强自我，而非取代自我。共勉。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 1.6万

留言

写留言

还没有留言，来写首评